Arrays.java

```java
//   ============***********  ARRAYS (Part -- 1)   **************================
//

// Arrays : List Of Elements Of the same type placed in a continous memory location

// Creating an Array
// data Type arrayName [] = new dataType[size];

import java.util.*;

public class Arrays {
    public static void main(String[] args) {

        // ==> Creating an array
        /*
        int marks[] = new int[50];
        int numbers [] = {1,2,3};
        int moreNumbers [] = {4,5,6};
        String fruits [] = {"apple","mango","orange"};
        */

        // ===> Input/Output
        /*
        int marks [] = new int[3];
        Scanner sc = new Scanner(System.in);

        marks[0] = sc.nextInt();
        marks[1] = sc.nextInt();
        marks[2] = sc.nextInt();

        System.out.println("Phy : " +marks[0]);
        System.out.println("che : " +marks[1]);
        System.out.println("maths : " +marks[2]);

        marks[2] = marks[2] +1;
        System.out.println("maths : " +marks[2]);

        System.out.println("Length Of Array : " + marks.length);      // For Array Length
        */

        // Arrays by as Arrgument ===> Array is a by Refrence Passing Arrgument, Bewcause They are a changing in other function by refrence a main function

        // ===> Linear Search in Array

        /*
        int num [] = {2,4,6,8,10,12,14,16,18,20};
        int key = 16;
```

```
47
48          int index = linear(num, key);
49          if (index == -1){
50              System.out.println("Not Found");
51          }
52          else{
53              System.out.println("Key is at index : " + index);
54          }
55          */
56
57      }
58      // ---------> Linear Search Another Function
59      /*
60      public static int linear(int num[], int key) {
61          for(int i=0; i<=num.length; i++){
62              if (num[i] == key){
63                  return i;
64              }
65          }
66          return -1;
67      }
68      */
69  }
70
71   // ===> Largest Element Search in Array
72  class Largest_Element{
73      public static int Largest(int num[]){
74          int largest = 0;                // Integer.MIN_VALUE = -Infinity
75       // int smallest =                     Integer.MAX_VALUE = +Infinity
76          for(int i=0; i<num.length; i++){
77              if(largest < num[i]){
78                  largest = num[i];
79              }
80          }
81          return largest;
82      }
83      public static void main(String[] args) {
84          int num [] = {8,5,7,23,45,140};
85          System.out.println("Largest Value is :" + Largest(num));
86      }
87  }
88
89  // ===> Binary Search Search in Array
90  class Binary_Search{
91      public static int Search(int nums[], int key) {
92          int start = 0;
93          int end = nums.length-1;
94          while(start<=end){
95              int mid = start+(end-start)/2;                      // int mid =
    start+end/2;
96              if(nums[mid] == key){
```

```
97                  return mid;
98              }
99              else if(nums[mid] < key){
100                 start = mid+1;
101             }
102             else{
103                 end = mid-1;
104             }
105         }
106         return -1;
107     }
108     public static void main(String[] args) {
109         int nums[] = {4,8,11,54,87,104,120};
110         int key = 120;
111         int result = Search(nums, key);
112         System.out.println(result);
113     }
114 }
115
116  // ===> Reverse an Array
117
118  class Reverse_Array{
119     public static void main(String[] args) {
120         int[] num = {7777,888,99,1,22,333,4444,55555,666666};
121         reverse(num);
122
123         for(int i=0; i<=num.length-1; i++){
124             System.out.print(num[i] + " ");
125         }
126         System.out.println();
127     }
128     public static void reverse(int[] num) {
129         int start = 0;
130         int end = num.length-1;
131
132         while(start<end){
133             // Swap
134             int temp = num[end];
135             num[end] = num[start];
136             num[start] = temp;
137
138             start++;
139             end--;
140         }
141     }
142 }
143
144  // ===> pairs in Array                    // ****** pairs of n = n(n-1)/2 ******** //
145
146  class Pairs_in_array{
147     public static void main(String[] args) {
```

```java
        int array[] = {1,3,5,7,9};
        pairs(array);
    }
    public static void pairs(int array[]) {
        int tp = 0;
        for(int i=0; i<=array.length-1; i++){
            for(int j=i+1; j<=array.length-1; j++){
                System.out.print("(" + array[i] + "," + array[j] + ") ");
                tp++;
            }
            System.out.println();
        }
        System.out.println("Total Pairs " + tp);
    }
}

// ===> Subarrays                    // ****** n of subarray = n(n+1)/2  ******* //
//                  : A continous part of array

class Sub_Arrays{
    public static void main(String[] args) {
        int[] nums = {2,4,6,8,10};
        Subarrays(nums);
    }
    public static void Subarrays(int[] nums) {
        int sa = 0;
        for(int i=0; i<=nums.length-1; i++){
            for(int j=i; j<=nums.length-1; j++){
                for(int k=i; k<=j; k++){
                    System.out.print(nums[k] + " ");
                }
                sa++;
                System.out.println();
            }
            System.out.println();
        }
        System.out.println("Total Sub Arrays is : " + sa);
    }

}


// ================**********   Arrays (Part-1) Complete  **********================ //

/*-------------------------------------------------------------------------------
*/

// ================**********   Arrays (Part-||)   **********================ //

// ===> Max Subarray Sum - |        (Brute Force)

```

```java
class max_subarray_sum_B{
    public static void main(String[] args) {
        int[] nums = {2,4,6,8,10};
        Subarrays(nums);
    }
    public static void Subarrays(int[] nums) {
        int currsum = 0;
        int maxsum = Integer.MIN_VALUE;
        for(int i=0; i<=nums.length-1; i++){
            for(int j=i; j<=nums.length-1; j++){
            currsum = 0;
                for(int k=i; k<=j; k++){
                    currsum+= nums[k];
                }
                System.out.println(currsum);
                if(maxsum<currsum){
                    maxsum = currsum;
                }
            }
        }
        System.out.println();
        System.out.println("maxsum is : " + maxsum);
    }
}

// ===> Max Subarray Sum - ||        (Prefix sum)
class max_subarray_sum_P{
    public static void prefix(int[] nums) {
        int currsum = 0;
        int maxsum = Integer.MIN_VALUE;

        // Create a Prefix Array
        int[] prefix = new int[nums.length];

        prefix[0] = nums[0];
        // Calculate Prefix Sum
        for(int i=1; i<=prefix.length-1; i++){
            prefix[i] = prefix[i-1] + nums[i];
        }

        for(int i=0; i<=nums.length-1; i++){
            int start = i;
            for(int j=i; j<=nums.length-1; j++){
                int end = j;
                if(start == 0){
                    currsum = prefix[end];
                }else{
                    currsum = prefix[end] - prefix[start-1];
                }
                if(maxsum < currsum){
                    maxsum = currsum;
```

```
                }
            }
        }
        System.out.println("Max Sum is : " + maxsum);
    }
    public static void main(String[] args) {
        int[] nums = {1,-2,6,-1,3};
        prefix(nums);
    }
}

// ===> Max Subarray Sum - |||       (Kadane's Algorithm)

class max_subarray_sum_K{
    public static void kadanes(int[] nums) {
        int currsum = 0;
        int maxsum = Integer.MIN_VALUE;

        for(int i=0; i<=nums.length-1; i++){
            currsum = currsum + nums[i];
            if(currsum < 0){
                currsum = 0;
            }
            maxsum = Math.max(currsum, maxsum);
        }
        System.out.println("Our Max Sum is : " + maxsum);
    }
    public static void main(String[] args) {
        // int[] nums = {-2,-3,4,-1,-2,1,5,-3};
        int[] nums = {1,-2,6,-1,3};
        kadanes(nums);

    }
}

// ===> Trapping RainWater

class trapping_rainWater{
    public static int trappedWater(int[] nums){
        int n = nums.length;
        // LeftmaxBoundry
        int[] leftMax = new int[n];
        leftMax[0] = nums[0];
        for(int  i=1; i<n; i++){
            leftMax[i] = Math.max(nums[i] , leftMax[i-1]);
        }

        // RightmaxBoundry
        int[] rightMax = new int[n];
        rightMax[n-1] = nums[n-1];
        for(int i=n-2; i>=0; i--){
```

```java
            rightMax[i] = Math.max(nums[i] , rightMax[i+1]);
        }

        // loop
        int trapped = 0;
        for(int i=0; i<n; i++){
            // Waterlevel = min(leftmax bound , rightMax bound)
            int waterlevel = Math.min(leftMax[i] , rightMax[i]);
            // trappedWater = waterlevel - height[i]
            trapped += waterlevel - nums[i];
        }
        return trapped;
    }
    public static void main(String[] args) {
        int[] nums = {4,2,0,6,3,2,5};
        System.out.println(trappedWater(nums));
    }
}

// Best time to Buy and Sell Stocks


class buy_and_sell_stocks{
    public static int buyAndSellStocks(int[] prices) {
        int buy = Integer.MAX_VALUE;
        int maxProfit  = 0;

        for(int i=0; i<=prices.length-1; i++){
            if(buy < prices[i]){
                int profit = prices[i] - buy;
                maxProfit = Math.max(maxProfit,profit);
            }else{
                buy = prices[i];
            }
        }
        return maxProfit;
    }
    public static void main(String[] args) {
        int[] prices = {7,1,5,3,6,4};
        int result = buyAndSellStocks(prices);
        System.out.println(result);
    }
}

// ================**********   Arrays (Part-||) Complete  **********================ //
```